Lecture 5:
I Hope You Like 2031

How and why to implement state machines and Real Time Operating Systems (RTOS).

Robojackets Firmware Training 2026

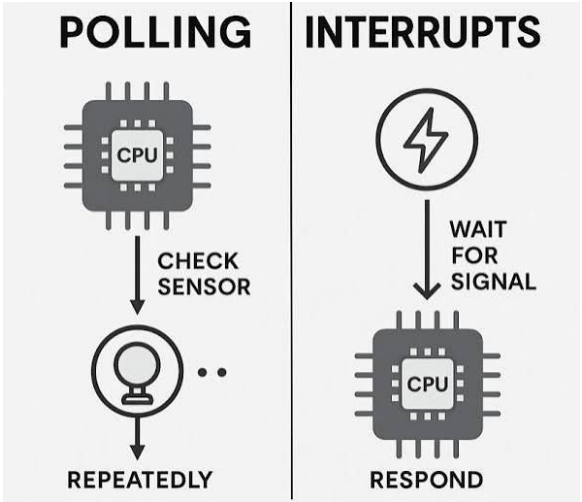
Administrivia - Training Outline

- Lecture 1: C/C++ Basics
- Lecture 2: Interrupts & Analog I/O
- Lecture 3: Communication Protocols
- Lecture 4: Driver Development
- **Lecture 5: State Machines & RTOS (today)**
- Practical Soldering Training

Missed Topics

Polling vs Interrupts

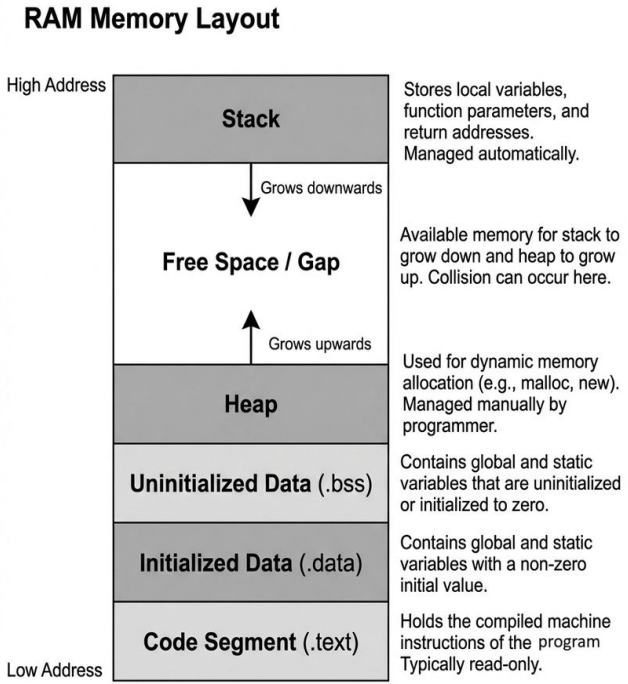
Polling is regularly and repeatedly checking for a change while interrupts are triggered on change and are only handled on this change.



- Polling
  - Constantly checks even when not needed.
  - Energy intensive
  - Ensure informations is up to date at a fixed time
  - Can check basically anything
- Interrupt
  - Only occurs when needed
  - Lower energy cost
  - Requires a specific change condition to occur. This is limited to what you MCU can detect.

Memory Layout

- Stack
  - Filled out by the compiler and CPU in a known layout
  - Stores per function information such as:
    - Local variables
    - Return address
    - Parameters
- Heap
  - Dynamically allocated by the program, thus **unknown** size
  - Dependant on the programmer to allocate and deallocate
  - Prone to “heap fragmentation”
    - The actual location in the heap is not guaranteed and can be inefficient (leave gaps).
    - Example: malloc 32kb of heap twice on a 64kb heap
      - Fails when the first malloc doesn’t start align
  - DO NOT USE IN FIRMWARE
    - Normal software has many more safety nets for malloc to fall on. Microcontrollers do not.
      - MUCH smaller memory (fragmentation more likely)
      - Process runs infinitely (missed dellocations never caught)

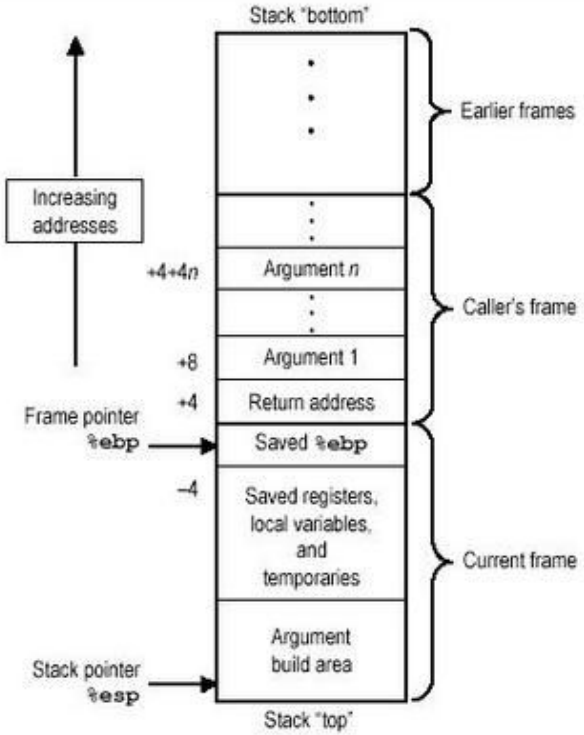


Stack Layout

All information for every function is stored in a “stack frame.” This is all stored relative to a “frame pointer.”

- Arguments
  - Parameter inputs for the function
- Return address
  - Pointer to where to store the return value
- Saved FP
  - Callers FP so that it can be restored on return
- Local Values
  - Local variables
- Stack Pointer
  - Points to the top of the stack

Note: While the stack does not suffer from fragmentation, it is still limited in size. To deep of function calls or recursive functions can cause it to run out of space and crash.



Cache

On the timescale of the CPU, memory is slow. Physical storage is astronomically slow.

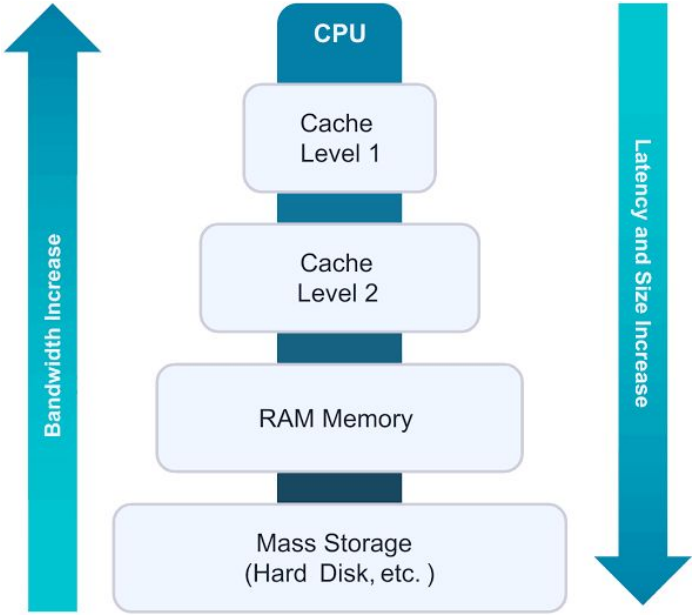
- RAM access 5-10 cycles
- External Storage access 1000s of cycles

To avoid wasting cycles reading storage, CPUs keep a copy of the data in a closer storage called cache.

- Cache access 1-2 cycles

This secondary/tertiary level of storage comes with risks that fall under the “cache coherence” problem.

Similar to the reasons we use “volatile”



Watchdog Timers

A watchdog is a process separate to another process which watches that process.

- This most commonly uses a timer to check how long the process takes
  - If the process stalls, dies, or never starts the watchdog will trigger
- There are often built in watchdog timers built into microcontrollers/HALs
  - However this is often limited to internal processes and are sometimes limited to resetting the MCU on trigger
- External device watchdogs
  - Specify some “pulse” to watch for, this depends how configurable the device is
  - Periodically check if this pulse has occurred in a time frame and if not handle the error

USB Protocol

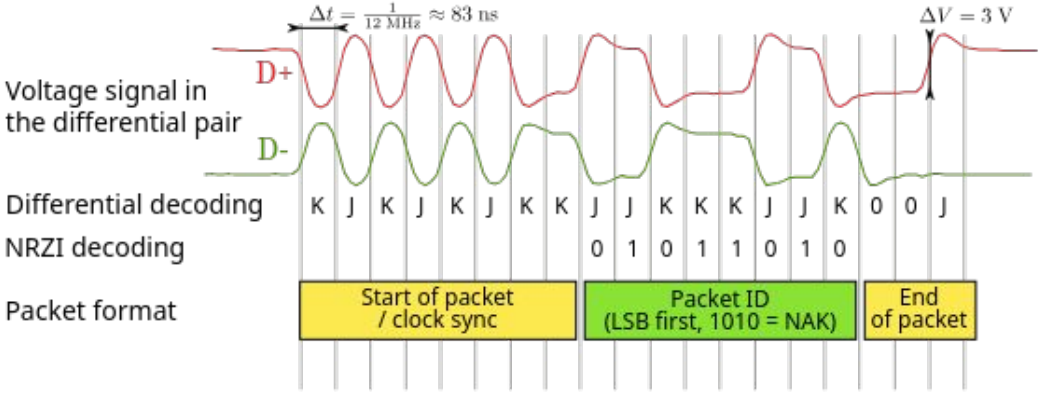
The USB communication protocol is an typically asynchronous serial protocol.

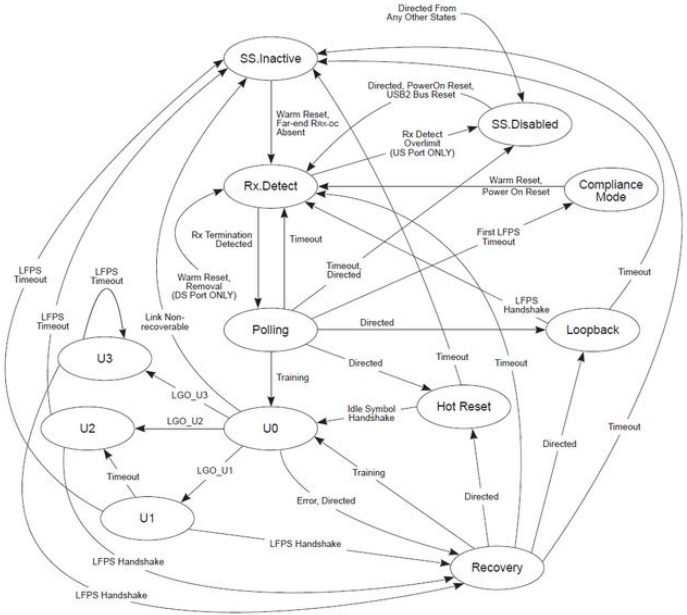
Like CAN, USB is a differential pair protocol.

It is far more complex and diverse than traditional embedded communication protocols.

Pros/Cons

- 1.5Mb/s - 80 Gb/s
- Relatively reliable
- Very complex



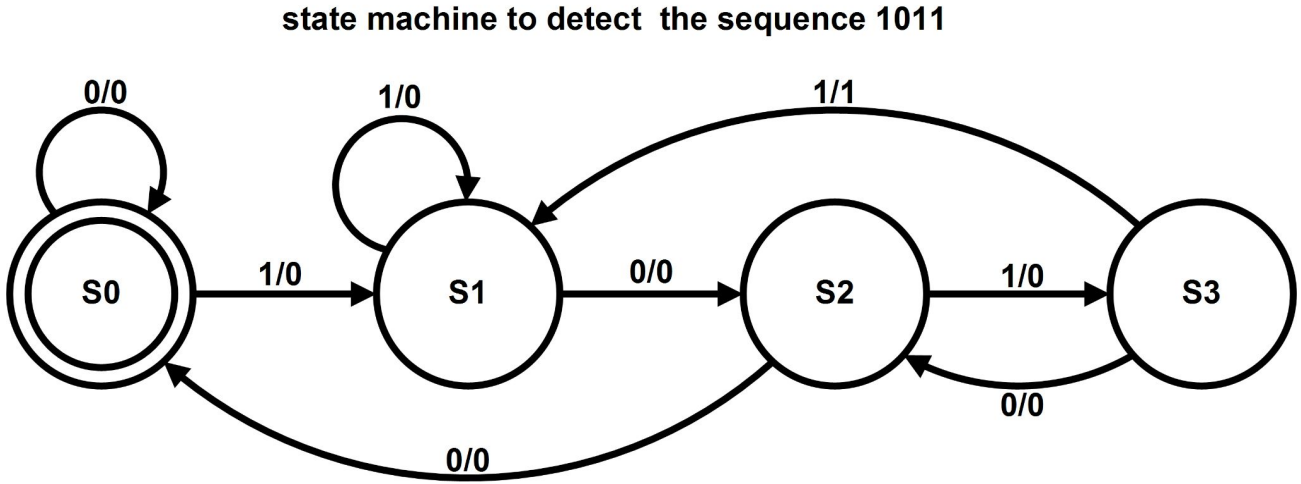


State Machines

States and Transitions

A state machine is a graph of states and transitions between them.

- State
  - A progress point
- Start state (Reset state)
  - The entry point to the state machine
- Transition
  - A decision on when/if to change states



State Transition Table

State transition tables are a text based formatting of state machines

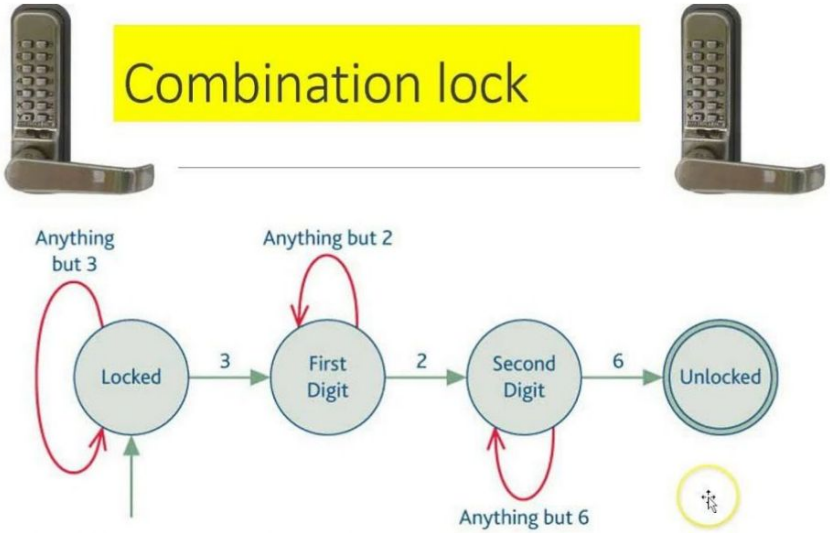


Fig. 1.3.3 *State transition diagram for a combination lock*

s	a	s'
Locked	Anything but 3	Locked
Locked	3	First Digit
First Digit	Anything but 2	First Digit
First Digit	2	Second Digit
Second Digit	Anything but 6	Second Digit
Second Digit	6	Unlocked

Applications

Almost all firmware more complicated than a sensor reader requires a system of states. Even at its simplest, startup -> main program -> main program etc is the minimum state machine.

However, proper state machines are vital to a lot of applications where you need to do different things depending on the state of the device.

Overall, this greatly simplifies control flow.

```
// Enum classes can have their sizes specified
// And are guaranteed unique
enum class RobotState : uint8_t {
    Idle = 'I',
    Main = 'M',
    Test = 'T',
    Error = 'E'
};

bool startup_tasks();
RobotState idle_task();
RobotState control_loop();
RobotState test_suite();
RobotState error_handler();

RobotState state = RobotState::Idle;

void setup() {
    if (!startup_tasks()) {
        state = RobotState::Error;
    }
}

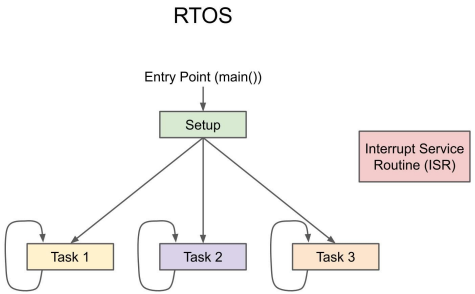
void loop() {
    switch (state) {
        case RobotState::Idle:
            state = idle_task();
        break;
        case RobotState::Main:
            state = control_loop();
        break;
        case RobotState::Test:
            state = test_suite();
        break;
        default:
            state = error_handler();
        break;
    }
}
```

Real Time Operating Systems (RTOS)

What is RTOS?

RTOS is an incredibly simplified operating system designed for embedded applications which does little more than manage task threads.

- Much stricter timing constraints than traditional OSs
  - While minor latencies are acceptable in a normal computer, embedded applications require much faster reaction times to stimuli
- Only one process
  - While traditional OSs have to balance many processes, we only ever work with one process
- Much less overhead
  - Scheduling, memory management, and other additional tasks in traditional OSs take up a lot of CPU time and memory.



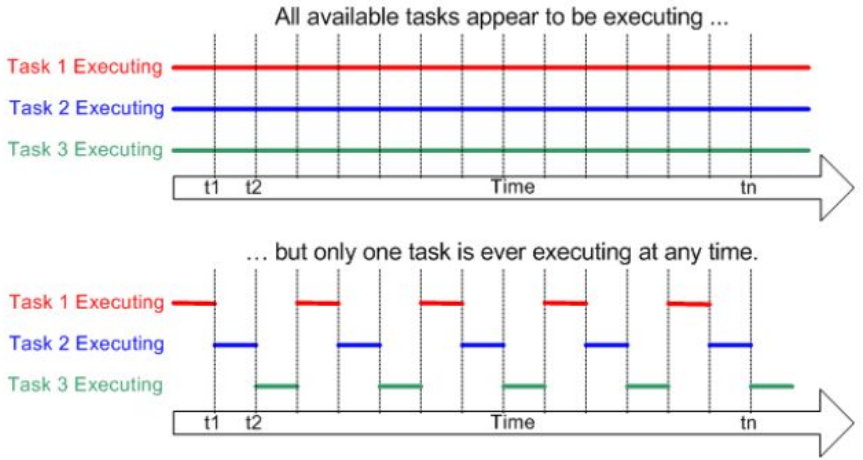
Threads

Threads are just a section of code, like a function. However, unlike normal inline code, multiple threads can be running at once (concurrency). This works by having a scheduler manage these threads, giving them CPU time and other resources as needed and available.

Even if you only have 1 CPU, it can make it appear as though you have more.

Thread vs Process

- Threads
  - Subset of a process
  - Shared memory
- Processes
  - Independent program in execution
  - Separate virtual memory



Shared Resources

Because threads can be scheduled non-deterministically, this can result in shared resources being accessed non-deterministically. This can lead to a “Data Race.”

A data race occurs when two or more threads try to access a shared memory location with at least one of those accesses being a write.

```
uint8_t x = 0;
uint8_t y = 10;

void t1() {
    x += 5;
    y = x;
}

void t2() {
    x = y;
    x++;
}
```

What are potential values of x and y after the two threads complete?

This process uses mutexes to protect data accesses. How does it still fail correctness if more than one thread is running?

```
#define MAX_QUEUE_SIZE 10

queue_t queue;
mutex_t q_lk;

bool check_size(queue_t* q) {
    bool too_big;
    acquire(&q_lk);
    too_big == q->size <= MAX_QUEUE_SIZE;
    release(&q_lk);
    return too_big;
}

void thread() {
    while (1) {
        if (check_size(&queue)) continue;

        uint8_t data = read_sensor();

        acquire(&q_lk);
        queue.push(data);
        release(&q_lk);
    }
}
```

Tasks

The key benefit of an RTOS is the task scheduler. Instead of managing complex state machines, manual timers, and the various importances of the jobs you need done, you just make tasks.

```
void setup() {
  // Initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
  while (!Serial) {
    ; // Wait for serial port to connect (needed for native USB boards)
  }

  // Create two tasks that will run independently
  xTaskCreate(
    TaskBlink,          // Pointer to the actual function handling the task
    "Blink",            // A text name given to the task for debugging
    128,                // Stack size in words (128 words = 256 bytes on UNO)
    NULL,               // Parameter passed into the task (not used here)
    1,                  // Task Priority (1 is low, higher numbers mean higher p
    NULL                // Task handle (not used here)
  );

  xTaskCreate(
    TaskSerialPrint,    // Function name
    "SerialPrint",      // Task name
    128,                // Stack size
    NULL,               // Parameters
    1,                  // Priority
    NULL                // Task handle
  );

  // The FreeRTOS task scheduler automatically starts after setup() finishes
}
```

```
void loop() {
  // Keep empty! FreeRTOS manages everything through tasks.
}

/*--------------------------------------------------*/
/*---------------------- Tasks ---------------------*/
/*--------------------------------------------------*/

void TaskBlink(void *pvParameters) {
  (void) pvParameters; // Prevent compiler warning for unused parameter

  pinMode(LED_BUILTIN, OUTPUT);

  for (;;) { // FreeRTOS tasks must run in an infinite loop and never return
    digitalWrite(LED_BUILTIN, HIGH);   // Turn the LED on
    // Use vTaskDelay instead of delay(). It yields CPU control to other tasks
    vTaskDelay(500 / portTICK_PERIOD_MS);

    digitalWrite(LED_BUILTIN, LOW);    // Turn the LED off
    vTaskDelay(500 / portTICK_PERIOD_MS); // 500ms delay
  }
}

void TaskSerialPrint(void *pvParameters) {
  (void) pvParameters;

  for (;;) {
    Serial.println("Hello from the FreeRTOS Serial Task!");
    vTaskDelay(1000 / portTICK_PERIOD_MS); // Wait for 1 second
  }
}
```

What is RTIC?

RTIC is Real Time Interrupt-driven Concurrency. It is very similar to RTOS, however it relies on hardware scheduling rather than software.

By relying on timer and external interrupts to trigger tasks rather than a scheduler, better performance can be achieved.

```
#[task(binds = UART0, local = [times: u32 = 0])]
fn uart0(cx: uart0::Context) {
    // Safe access to local `static mut` variable
    *cx.local.times += 1;

    hprintln!(
        "UART0 called {} time{}",
        *cx.local.times,
        if *cx.local.times > 1 { "s" } else { "" }
    );
}
```

For example this task is only triggered by a UART line interrupt.

*Note that RTIC is exclusive to Rust.

LAB TIME!!!!!

Todays Lab:

- **State Machine**
  - Create a digital lock that can be unlocked using the two buttons
  - Use a state machine to accomplish this